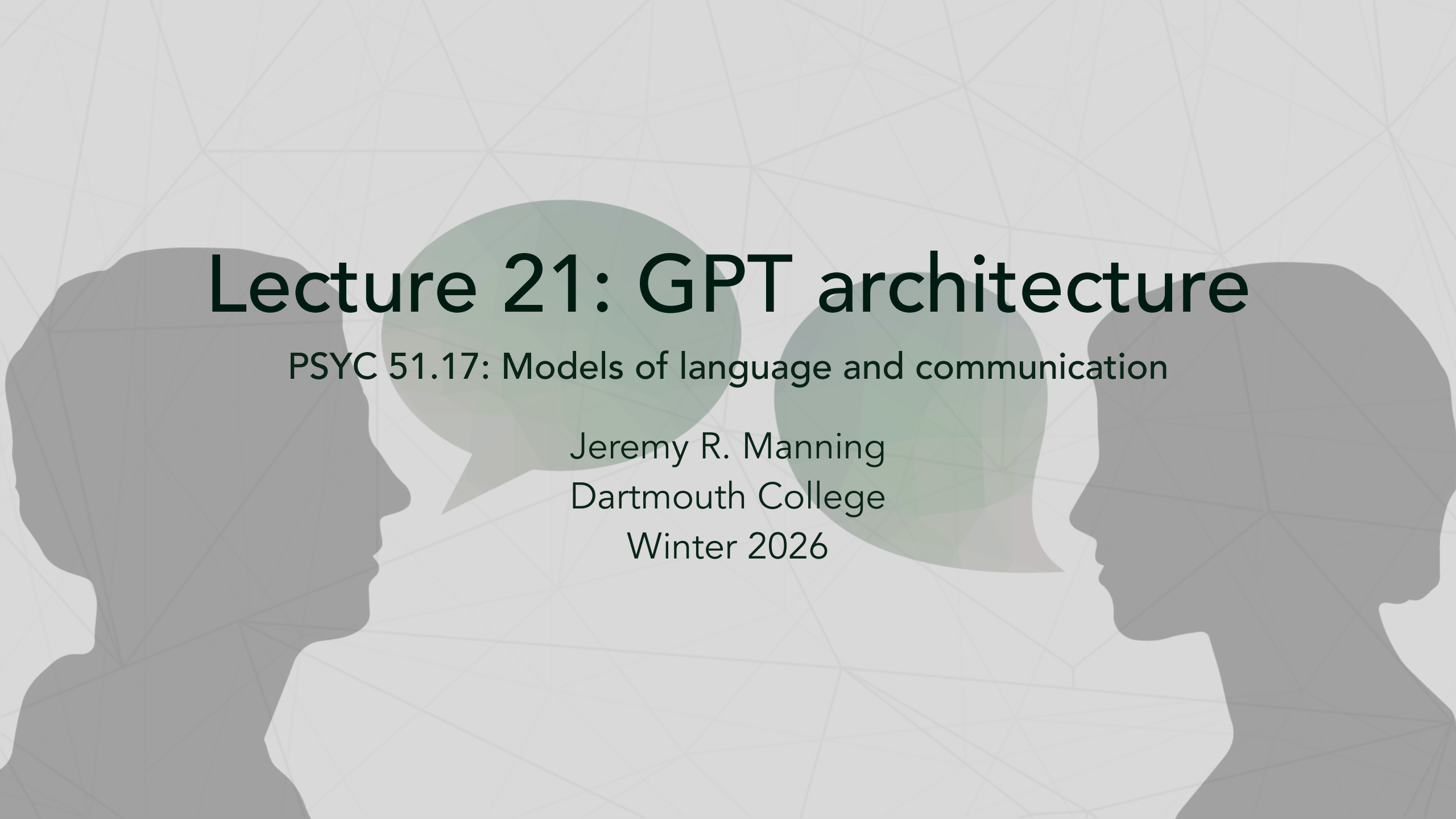




Lecture 21: GPT architecture

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will be able to...

1. Explain the **generative pre-training** paradigm and why it was revolutionary
2. Describe the **transformer decoder** architecture used in GPT
3. Distinguish between **pre-training** and **fine-tuning** stages
4. Compare GPT's **autoregressive** approach with BERT's **bidirectional** approach
5. Identify how the decoder stack has evolved from GPT-1 to modern LLMs (2024)

Announcements

Assignment 4 due this week

Assignment 4 (Customer Service Chatbot) is due **February 16, 11:59 PM EST.**

Final project and optional assignment

- **Final Project** has been released! Due **March 9, 11:59 PM EST**
- **Assignment 5** (Build GPT) is now available as **optional extra credit**

From BERT to GPT

Two philosophies of language modeling

- **BERT** (encoder): Mask random tokens, predict them using **bidirectional** context. Best for *understanding* tasks (classification, NER, QA).
- **GPT** (decoder): Predict the **next token** using only **left-to-right** context. Natural for *generation* tasks, but also capable of understanding.

Questions to consider

Both models were published in 2018. BERT initially dominated benchmarks, but GPT's approach proved more scalable. Why might autoregressive modeling have a higher ceiling?

The generative pre-training paradigm

Radford et al. (2018): 'Improving Language Understanding by Generative Pre-Training'

GPT introduced a two-stage approach that changed NLP forever:

1. **Pre-train** on massive unlabeled text (unsupervised)
2. **Fine-tune** on specific downstream tasks (supervised)

Why this was revolutionary

- **Before GPT:** Train task-specific models from scratch, needing large labeled datasets for each task
- **After GPT:** Learn general language representations once, then adapt cheaply to any task
- This is **transfer learning** for NLP — the same idea that transformed computer vision with ImageNet

The transformer decoder

GPT's architecture

GPT uses only the **decoder** half of the original transformer ([Vaswani et al., 2017](#)). The data flows through:

1. **Token embeddings**: Map each token to a 768-dimensional vector
2. **Position embeddings**: Add learned positional information
3. **N transformer blocks**: Each contains masked self-attention + feed-forward network
4. **Output head**: Project back to vocabulary size for next-token prediction

Key difference from the full transformer

The original transformer has both an encoder and a decoder with cross-attention between them. GPT removes the encoder entirely and removes cross-attention — it only uses **masked self-attention** within the decoder.

Masked (causal) self-attention

Recall from Lecture 15

In standard self-attention (Lecture 15), every token attends to every other token. In **causal** (masked) self-attention, each token can only attend to tokens at **earlier positions** (and itself). This prevents the model from "peeking" at future tokens during training.

Attention pattern for 'The cat sat on'

	The	cat	sat	on
The	1.0	-inf	-inf	-inf
cat	0.3	0.7	-inf	-inf
sat	0.2	0.3	0.5	-inf
on	0.1	0.2	0.3	0.4

The **-inf** entries become 0 after softmax, ensuring no information flows from future tokens. The causal mask enables both parallel training (all positions in one pass) and autoregressive generation (left-to-right, one token at a time).

GPT-1 specifications

Model architecture

Component	Value
Transformer layers	12
Hidden size (d_model)	768
Attention heads	12
Max sequence length	512 tokens
Vocabulary size	40,000 (BPE)
Feed-forward size	3,072 (4×768)
Total parameters	~117 million

Training details

Detail	Value
Training data	BooksCorpus (~7 000 books)

Stage 1: Pre-training

The pre-training objective

Maximize the likelihood of each token given all preceding tokens:

$$\mathcal{L}_{\text{pre-train}} = \sum_{i=1}^N \log P(t_i \mid t_1, t_2, \dots, t_{i-1}; \Theta)$$

This is simply **next token prediction** over a large corpus. No labels needed.

Training on 'The quick brown fox jumps'

The model learns from every position simultaneously:

- Input: "The" → Target: "quick"
- Input: "The quick" → Target: "brown"
- Input: "The quick brown" → Target: "fox"
- Input: "The quick brown fox" → Target: "jumps"

Every text passage provides training signal at every token position.

Stage 2: Fine-tuning

Adapting to downstream tasks

After pre-training, GPT is adapted to specific tasks by:

1. **Reformatting** task inputs with special delimiter tokens
2. **Adding** a small classification head (linear layer) on top
3. **Training** with both task loss and a language modeling auxiliary loss

$$\mathcal{L}_{\text{fine-tune}} = \mathcal{L}_{\text{task}} + \lambda \cdot \mathcal{L}_{\text{LM}}$$

The auxiliary LM loss ($\lambda = 0.5$) improves generalization and speeds convergence.

Task formatting

All tasks become text completion: `[START] text [DELIM]` → predict label.

Classification, entailment, similarity, and QA all use the same architecture with different input formats. This unifying insight — every NLP task as text completion — is why pre-training transfers so effectively.

Fine-tuning example: sentiment classification

Classifying movie reviews

```
1  # Format input for GPT
2  text = "This movie was absolutely fantastic!"
3  formatted = f"[START] {text} [DELIM]"
4
5  # Tokenize and pass through GPT
6  token_ids = tokenizer.encode(formatted)
7  hidden_states = gpt_model(token_ids) # (1, seq_len, 768)
8
9  # Take hidden state at the last (DELIM) position
10 final_hidden = hidden_states[0, -1, :] # (768,)
11
12 # Pass through classification head
13 classification_head = nn.Linear(768, 2) # 2 classes
14 logits = classification_head(final_hidden) # [pos_score,
15 neg_score]
```

continued...

Fine-tuning example: sentiment classification

Classifying movie reviews

```
16 # Compute loss
17 loss = cross_entropy(logits, label_id) # label_id = 0 (positive)
```

...continued

GPT-1 results

Performance on standard benchmarks

Task	Previous SOTA	GPT-1
Question answering	86.7	88.1
Semantic similarity	85.0	85.8
Text classification	93.0	94.2
Natural language inference	80.6	82.1

GPT-1 achieved state-of-the-art on **9 out of 12** benchmark tasks.

The key finding

The largest improvements came on tasks with **less training data**. Pre-training provided a strong prior that compensated for limited labeled examples — exactly the promise of transfer learning.

Zero-shot and few-shot learning

Learning without (much) fine-tuning

- **Zero-shot:** No task-specific examples. The model must generalize purely from its pre-training.
- **Few-shot:** A small number of examples (1–10) provided as context in the prompt.
- **Fine-tuning:** Full supervised training on a task-specific dataset.

GPT-1's limitation

GPT-1's zero-shot performance was **weak**. The model had learned rich language representations but struggled to apply them without explicit task formatting. This motivated the development of GPT-2 and GPT-3, which aimed to make models that could perform tasks *without* fine-tuning.

GPT vs BERT

Head-to-head comparison

Aspect	BERT (encoder)	GPT (decoder)
Training objective	Masked language model	Autoregressive LM
Context direction	Bidirectional	Left-to-right
Best suited for	Understanding (NER, QA, classification)	Generation (text, code, dialogue)
Fine-tuning approach	Task-specific heads	Unified sequence format
Base parameters	110M	117M
Tokens predicted per pass	~15% (masked only)	100% (every position)

The scalability argument

BERT dominated understanding benchmarks in 2018–2019, but GPT's autoregressive approach proved more scalable. Why? Predicting *every* token gives $6.7\times$ more training signal per pass, and generation is a superset of understanding — a model that can *write* coherent text implicitly *understands* it.

The modern decoder stack (2024)

Every component has been upgraded since GPT-1

Component	GPT-1 (2018)	Modern LLMs (2024)	Why the change
Normalization	<u>LayerNorm</u>	<u>RMSNorm</u>	10–15% faster, no mean computation
Position encoding	Learned absolute	<u>RoPE</u>	Extrapolates to unseen lengths
Activation	GELU	<u>SwiGLU</u>	~1% better across benchmarks
Attention	Multi-head (MHA)	<u>Grouped-query (GQA)</u>	2× faster inference, same quality

The takeaway

The *conceptual* architecture is the same: token embeddings → causal attention → FFN → output head. But every piece has been systematically optimized. Modern LLMs like LLaMA 3, Gemma 2, and Mistral all use this upgraded stack.

Multi-token prediction

Predicting more than one token at a time (Meta/FAIR, 2024)

Standard GPT predicts one token ahead. Multi-token prediction trains the model to predict the **next 2–4 tokens simultaneously** using independent output heads sharing the same backbone.

Why this matters

Benefit	Explanation
Better representations	Forces the model to plan ahead, not just match local patterns
Faster inference	Can decode 2–4× faster with speculative decoding
Stronger coding	12% improvement on code generation (HumanEval), where planning matters most

The connection to how humans process language

Humans don't process language one word at a time — we predict upcoming words *in chunks*.

Hybrid architectures: attention meets state-space models

Not everything needs to be a transformer

Jamba (AI21, 2024) interleaves Transformer attention layers with Mamba state-space layers:

- **Attention layers:** Good at precise retrieval ("what was the third item?")
- **Mamba layers:** Good at long-range compression and fast inference ($O(n)$ vs $O(n^2)$)
- **Hybrid:** Gets the best of both — 256K context at 3× the throughput of pure Transformers

Questions to consider

The Transformer has dominated since 2017. But Jamba, Mamba-2, and RWKV suggest that the optimal architecture may be a *hybrid*. What does it mean that different computational primitives (attention vs. recurrence) excel at different aspects of language?

Limitations of GPT-1 and the path forward

What GPT-1 could not do well

- **Weak zero-shot performance:** Required fine-tuning for each new task
- **Small model:** 117M parameters (small by modern standards)
- **Limited data:** Trained on ~5B tokens from BooksCorpus only
- **Short context:** Maximum sequence length of 512 tokens
- **Hallucinations:** Confidently stated incorrect facts

The path forward

Each limitation suggested a clear direction: bigger models, more data, longer contexts, better training. GPT-2 and GPT-3 would systematically address these limitations through **scale** — but as we'll see in the next lecture, scale alone brings its own surprises and controversies.

Discussion

Questions to consider

1. **Autoregressive vs bidirectional:** GPT predicts left-to-right; BERT sees both directions. Humans process language incrementally (left-to-right in English) but with rich top-down expectations. Which model is more "brain-like"?
2. **The generation advantage:** GPT can *generate* text, BERT cannot. Is generation a prerequisite for understanding? Can you truly understand language if you can't produce it?
3. **Component upgrades:** Every piece of GPT-1 has been replaced (LayerNorm→RMSNorm, learned PE→RoPE, etc.) but the architecture is "the same." At what point does a ship become a different ship? (Theseus's paradox for neural networks.)
4. **Hybrid architectures:** Jamba mixes attention and state-space

Further reading

Further reading

Radford et al. (2018). "Improving Language Understanding by Generative Pre-Training" — The original GPT paper.

Radford et al. (2019). "Language Models are Unsupervised Multitask Learners" — GPT-2: larger models, zero-shot transfer.

Su et al. (2024, *Neurocomputing*). "RoFormer: Enhanced Transformer with Rotary Position Embedding" — RoPE, now standard in most LLMs.

Ainslie et al. (2023, *EMNLP*). "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints" — Grouped-query attention.

Gloeckle et al. (2024, *arXiv*). "Better & Faster Large Language Models via Multi-token Prediction" — Meta/FAIR multi-token prediction.

Lieber et al. (2024, *arXiv*). "Jamba: A Hybrid Transformer-Mamba Language Model" — Hybrid attention + SSM architecture.

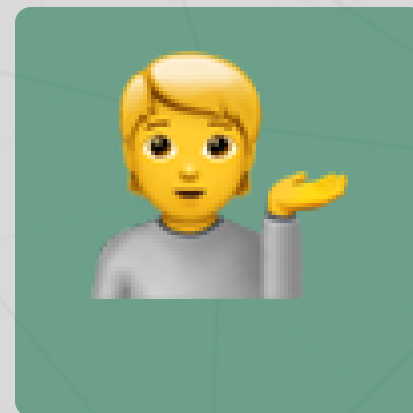
Questions?



Email



Discord



Office hours

Up next...

Scaling up: from GPT-2 to GPT-4, emergent abilities, reasoning, and the path to ChatGPT